Axona

A Programmer's Introduction

David A. Smith · *Axona.net* · v0.1 · a ∼30-minute talk

A 30-minute tour for developers: what Axona is, how it works, what protects it, the API you actually call, and a live app — Axona Minimal — built in about sixty lines. Follow along at `demo.axona.net`.

I What Axona is

Axona is a peer-to-peer substrate for **addressing, routing, and publish/subscribe** that runs with no platform in the middle — including in an ordinary web browser.[1] You get a global pub/sub bus where any peer can publish to a topic and any peer can subscribe to it, and the messages flow peer-to-peer over an encrypted mesh.

Three things make it different from a hosted message broker:

- **Self-authenticating.** A peer's identity *is* its cryptographic keypair; its node address is derived from its public key and verified directly by other peers. There is no account, no certificate authority, no registrar.[2]
- **Geo-aware, not geo-fenced.** The top of every address is a coarse geographic cell, so routing favors locality — an "area code," not a wall.
- **No central chokepoint.** A bootstrap broker introduces new peers, but the mesh carries the traffic itself and converges its own state. The first version is live; anyone can connect.

[1] Pure JS/WASM. The reference peer is a web page; the same kernel runs in Node and in the simulator.

[2] You are your key. Nothing to sign up for, nothing to revoke through.

II How it works

Addresses. Every node has a 264-bit id. The *top 8 bits* are an S2 geographic cell (one of 192 covering the planet); the rest is SHA-256 of the node's public key.[3] Closeness is XOR distance, which the top byte dominates — so "nearby in the keyspace" tracks "nearby on Earth," *softly*: it biases routing without partitioning anyone out.

The mesh. In the browser, peers form a WebRTC mesh. A WebSocket *bridge* does the initial introduction (and relays signaling), but it is a bootstrap rendezvous, not a router — peers can even open new links through existing peers without it. Each node keeps a learned routing table (its *synaptome*) and reaches any id in a few hops.

[3] So the address is `geocell` ‖ `hash(pubkey)` — location-biased at the top, key-bound below. Topic ids are built the same way (see §V).

Pub/sub. A topic maps to an id; the **R** nodes closest to that id are its *root set* (replicas). A publish goes to the roots, which fan it out down a tree of sub-axons to the subscribers.[4] Three mechanisms keep replicas consistent — gap-safe replay on (re)subscribe, root-to-root anti-entropy, and kill convergence — so a message is backfilled rather than lost, and a retraction stays retracted.

[4] Replication + a fan-out tree is what makes delivery survive churn: lose a root or a branch and the others still carry the message.

III The security model

Everything below is enforced by cryptography the peers carry themselves — no trust server sits behind any of it.

- **Identity = keypair.** The node id is derived from the public key, so a peer cannot claim to be another; the transport handshake is mutually authenticated and channel-bound (a man-in-the-middle that swaps the media fingerprint fails the bind).
- **Signed publishes, verified at ingress.** A signed message is checked against the publisher's key at the *root*, before it is cached or fanned out — spoofed-signature spam dies at the edge.
- **You can only act for yourself.** A subscribe/unsubscribe is honored only for the authenticated channel peer's own id, so no one can subscribe a victim (amplification) or unsubscribe them (silencing).
- **Bounded hosting.** A node only becomes a root for topics it is genuinely closest to, and inbound size/count caps apply — a flood of junk topics can't make every node allocate unbounded state.
- **Content-addressed + fresh.** The message id is the hash of its content; a per-publisher sequence and TTL drop stale or replayed envelopes.
- **Sybil / placement defense.** A memory-hard proof-of-work prices the *publish identity*, decoupled from the node address so it can't be used to grind a favorable position in the keyspace.

What it does *not* promise. A kill is best-effort redaction, not a cryptographic un-send — a peer that already holds the bytes keeps them. And a medium no platform can censor is, by the same token, one no platform can moderate. Design your app with that in mind.

IV The API you'll actually use

The whole surface is small. You construct a peer, then publish and subscribe.

`deriveIdentity({lat,lng})`	make a keypair + 264-bit id
`webTransport({bridgeUrl, identity})`	browser transport (mesh + bridge)
`new AxonaPeer({domain,node,identity,transport})`	the peer
`peer.start()` / `peer.leave()`	join / drain + leave
`peer.pub(topic, msg, {publisher})`	publish → `msgId`
`peer.sub(topic, handler, {publisher,since})`	subscribe → `Subscription`
`peer.unsub(topic, {publisher})`	leave a topic
`peer.pull(msgId | null, {topic,publisher})`	fetch by id, or latest
`peer.kill(topic, msgId, {publisher})`	retract (creator-only)
`peer.host(topic?)`	serve a topic without consuming it
`peer.metrics / peer.health()`	counters / introspection

Where a topic lives is set by `opts.publisher`: omit it to publish under *your own* feed, pass `null` for a well-known public topic, or pass a 66-char hex publisher to read/write *someone else's* feed. A shared "room" uses a region-anchored synthetic publisher so every peer in that region derives the same topic id.[5]

The envelope a subscriber receives is `{ msgId, seq, ts, topic, message, publisher, signerPubkey }`; a retraction arrives on the same handler as `{ deleted: true, msgId }`.

[5] The helper `resolveAnchor()` returns that publisher from `?region=` (default us-east), along with the lat/lng to seed your identity.

V Build-along: Axona Minimal

A topic field, a message field, a received-messages pane. Live at `demo.axona.net/apps/axona-minimal/` — open it in two tabs and watch a message cross between them. Here is the whole of it.

1. Connect — derive an identity, open the transport, build the peer, wait briefly for the mesh:

```
import { AxonaPeer, AxonaDomain, NeuronNode, deriveIdentity } from '/src/index.js';
import { webTransport } from '/src/transport/web/index.js';
import { resolveAnchor } from '../lib/region.js';

const BRIDGE = 'wss://bridge.axona.net';
const ANCHOR = resolveAnchor();                      // { name, center:{lat,lng}, publisher }

const identity  = await deriveIdentity({ lat: ANCHOR.center.lat, lng: ANCHOR.center.lng });
const transport = webTransport({ bridgeUrl: BRIDGE, identity });
const node      = new NeuronNode({ id: BigInt('0x' + identity.id), lat: ANCHOR.center.lat, lng: ANCHOR.center.lng });
node.transport  = transport;
const peer = new AxonaPeer({ domain: new AxonaDomain({ k: 20 }), node, identity, transport });

await transport.start(identity.id);
await peer.start();                                  // mesh forms over the next moment
```

2. Subscribe to the topic. Re-subscribing to a new topic drops the old one; the handler gets each envelope:

```
currentSub = await peer.sub(topic, (env) => {
  if (!env || env.deleted || seen.has(env.msgId)) return;   // skip our own echo
  seen.add(env.msgId);
  render(env.message, env.signerPubkey, /*self=*/false);
}, { publisher: ANCHOR.publisher, since: 'all' });
```

3. Publish. Own publishes may not echo back, so render optimistically and let the `seen` set dedup if they do:

```
const msgId = await peer.pub(topic, text, { publisher: ANCHOR.publisher });
seen.add(msgId);
render(text, null, /*self=*/true);
```

That is the entire networking story: `deriveIdentity` → `webTransport` → `AxonaPeer` → `sub`/`pub`. Everything else in the file is DOM glue.

VI Try it, and go deeper

- **Run it:** `demo.axona.net/apps/axona-minimal/` (and the other demos on that page). Add `?region=uswest` to land in another keyspace.
- **Read it:** the *Explainer* (how the routing works), the *Architecture* note (kernel · transport · bridge · wire), and the *API Reference* for every exported symbol.
- **Host it:** run a relay (`axona-relay`) to carry topics in your region, or stand up your own bridge — the only piece you need to bootstrap.

Sixty lines and a browser tab put you on the network. There is no step zero.